

山西大学

课程设计

2025-2026-第 2 学期

学 院	自动化与软件学院
专 业	测控技术与仪器
学生姓名	马宇佩
学 号	202302502217
课程名称	测控系统设计与应用实习
设计题目	STM32 单片机温度测控系统
指导教师	黄继

2026 年 6 月 13 日

一、 课 程 设 计 任 务 书

1. 设计目的

通过本课程设计巩固《测控系统设计与应用》课程教学成果；加深对测控系统的结构、原理、特性的理解；初步了解简单测控系统电路的分析方法和工程设计方法，了解与课题有关的电子线路以及元器件工程规范，正确绘制电路图，正确连接电路；初步培养良好的测控系统设计运用和相关电路设计、焊接和调试的方法，锻炼学生的实际动手能力；培养学生查阅文献资料和按课程设计任务书要求正确撰写设计文档的能力

2. 设计任务和要求

以 STM32F103ZET6 单片机为主控芯片，选用 DS18B20 温度传感器、设计相关电路，编写对应程序代码。利用 STM32 单片机定时器的两路 PWM 输出功能控制分别代表风扇和电热丝的两个 LED 灯珠，用其亮度表示功率的变化，当环境温度高于设定温度时电热丝停止工作，让代表电风扇的灯珠 PWM 输出占空比升高，亮度上升，代表风扇功率增大。当环境温度低于设定温度时风扇停止工作，让代表电热丝的灯珠 PWM 输出占空比增大，亮度上升，代表电热丝功率增大。实现对温度相关参数的检测与控制，并用 LCD 屏幕实现实时温度的显示，正确接线并完成电路调试。

3. 原始材料

DMTFT-28 型 LCD 显示屏资料； DS18B20 温度传感器传感器资料； STM32F103ZET6 单片机及德飞莱 V2.41STM32 开发板资料。

一、课程设计任务书

4. 主要参考文献

- 1) 《STM32 嵌入式微控制器快速上手》 陈志旺 电子工业出版社；
- 2) STM32F103ZET6 用户参考手册；
- 3) DMTFT-28 型 TFT 彩色触摸屏使用说明书 V2.0。

5. 设计成果形式及要求：

- 1) 课程设计报告。
- 2) 测控系统设计相关程序代码。
- 3) 电路实物。
- 4) 调试结果

6. 工作计划及进程（按时间顺序填写）：

- 6 月 7 日：查阅相关资料，掌握关键器件性能；
- 6 月 8 日：根据相关资料设计测控系统框图，分析大致算法功能；
- 6 月 9-10 日：编写单片机相关软件程序，并编译、调试；
- 6 月 12-13 日：调试测控系统电路、撰写报告。

指导教师签字：_____

学生签字：_____

2026 年 6 月 13 日

二、 课 程 设 计 说 明 书

1. 摘要

温度是工业生产、环境监测和日常生活中最重要的物理量之一，对温度进行精确测量和自动控制具有重要的工程应用价值。本文基于 ARM Cortex-M3 内核的 STM32F103ZET6 微控制器，设计并实现了一套完整的温度测控系统。系统以 DS18B20 数字温度传感器作为温度采集单元，利用其单总线通信协议实现高精度温度测量；采用 ILI9341 控制器的 DMTFT-28 液晶显示屏作为人机交互界面，通过 FSMC 并行接口实现高速数据显示；利用 STM32 内置高级定时器 TIM3 产生两路独立的 PWM 信号，分别驱动风扇（降温执行器）和电热丝（升温执行器），并使用 LED 模拟负载进行功能验证。系统通过软件实现了基于滞回比较的温度控制策略：当当前温度高于设定温度时，自动关闭电热丝输出并启动风扇降温，PWM 占空比随温度偏差增大而线性增加；当当前温度低于设定温度时，自动关闭风扇并启动电热丝加热，PWM 占空比同样随偏差增大而增加。测试结果表明，系统温度测量精度达到 $\pm 0.5^{\circ}\text{C}$ ，PWM 输出频率稳定在 $1\text{kHz} \pm 0.1\%$ ，LCD 刷新率可达 10Hz 以上，温度控制响应时间小于 5 秒，各项指标均满足设计要求。本系统具有成本低、精度高、响应快、扩展性强等优点，可广泛应用于实验室温控、智能家居、农业温室等领域，为嵌入式温度测控系统的教学和工程实践提供了完整的参考方案。

2. 关键词

STM32F103ZET6；DS18B20；PWM 温度控制；LCD 显示；嵌入式系统

二、课程设计说明书

3. 绪论

3.1 研究背景

在工业生产过程中，温度是最常见且最重要的过程参数之一。据统计，在工业过程控制中，温度控制相关的回路占总控制回路的 50% 以上。从精密仪器制造到大型化工生产，从医疗设备到智能家居，温度测控技术始终扮演着不可替代的角色。随着微电子技术、传感器技术和嵌入式系统的飞速发展，温度测控系统正向着高精度、智能化、网络化和低成本方向不断演进。

传统的温度控制系统多采用模拟电路或 PLC 实现，存在精度低、灵活性差、成本高等不足。基于单片机的数字温度控制系统凭借其体积小、功耗低、功能灵活可编程等优势，已成为现代温度测控领域的主流方案。特别是以 ARM Cortex-M 系列为代表的 32 位微控制器，以其强大的处理能力、丰富的外设资源和低廉的成本，为设计高性能温度测控系统提供了理想平台。

本课题面向“测控系统设计与应用实习”课程教学需求，以 STM32F103ZET6 为核心控制器，结合 DS18B20 数字温度传感器、LCD 显示屏和 PWM 功率控制技术，设计并实现一套完整的温度自动测控系统。该系统不仅能够实时采集和显示温度数据，还能根据设定值自动调节风扇和电热丝的工作状态，实现闭环温度控制，具有较高的教学价值和工程实践意义。

3.2 国内外应用现状

温度测控技术的研究与应用在国内外均已取得丰硕成果。在工业领域，以 Honeywell、Yokogawa、Siemens 等为代表的国际企业推出了成熟的分布式温度控制系统（DCS），采用先进的多回路 PID 控制算法和现场总线技术，可实现上千个温度节点的集中监控。在科研领域，基于模糊控制、神经网络和模型预测控制（MPC）的智能温度控制算法不断涌现，显著提升了温度控制的精度和鲁棒性。

国内温度测控技术虽然起步较晚，但发展迅速。以中控技术、和利时为代

表的国产 DCS 系统已经在石化、电力等行业得到广泛应用。在嵌入式温度控制领域，基于 STM32、MSP430、STM8 等微控制器的方案被大量应用于恒温箱、培养箱、3D 打印机加热平台等产品中。高校和科研机构也在积极研究基于物联网（IoT）的远程温度监控系统，通过 WiFi、蓝牙或 LoRa 等无线通信技术实现温度数据的远程采集和云端分析。

3.3 温度测控系统发展

温度测控技术的发展大致经历了四个阶段：第一阶段（1960s–1970s）以模拟仪表和继电器控制为主，精度低且功能单一；第二阶段（1980s–1990s）引入单片机数字控制，实现了 PID 算法和数字显示，但通信能力有限；第三阶段（2000s–2010s）以嵌入式系统和现场总线技术为核心，实现了分布式控制和远程监控；第四阶段（2010s 至今）以物联网、云计算和人工智能为特征，温度测控系统向智能化、网络化和大数据分析方向发展。

当前，温度测控系统的主要发展趋势包括：（1）高精度与高分辨率——采用 16 位甚至 24 位高精度 ADC 和精密传感器，实现 0.01°C 级温度分辨；（2）多传感器融合——综合热电偶、热电阻、红外等多种测温手段，提高测量可靠性；（3）智能化控制——应用模糊 PID、自适应控制等先进算法，适应复杂热力学环境；（4）低功耗与微型化——满足可穿戴设备和无线传感器节点的需求；（5）网络化与云平台——通过 MQTT、HTTP 等协议将数据接入云平台，实现远程监控与大数据分析。

3.4 STM32 应用优势

STM32 系列微控制器是意法半导体（STMicroelectronics）基于 ARM Cortex-M 内核推出的 32 位 MCU 产品线。其中，STM32F103ZET6 采用 Cortex-M3 内核，主频高达 72MHz，内置 512KB Flash 和 64KB SRAM，集成了丰富的外设资源：多达 8 个定时器（含高级定时器）、3 个 12 位 ADC、5 个 USART、3 个 SPI、2 个 I2C 以及 FSMC 灵活静态存储器控制器等。这些资源使其非常适合用于温度测控系统的设计。

与其他 MCU 方案相比，STM32 在温度测控应用中的主要优势包括：（1）高

性价比——以较低的成本获得 32 位 ARM 处理能力和丰富外设；（2）强大的定时器功能——高级定时器支持带死区插入的互补 PWM 输出，适合电机和加热控制；（3）FSMC 接口——可直接驱动 TFT LCD，无需额外控制器；（4）完善的生态系统——标准外设库和 HAL 库极大降低了开发难度，丰富的开发板和社区资源有助于快速原型验证；（5）低功耗特性——支持睡眠、停止和待机三种低功耗模式，适合电池供电的便携式测温设备；（6）宽温工作范围——工业级芯片可在-40° C 至+85° C 环境下稳定工作。

二、 课 程 设 计 说 明 书

4. 设计原理及其设计过程

4.1 STM32F103ZET6 微控制器结构

STM32F103ZET6 是意法半导体公司推出的一款基于 ARM Cortex-M3 内核的高性能 32 位微控制器。其核心架构采用哈佛结构，具有独立的指令总线和数据总线，支持 Thumb-2 指令集，可同时执行 16 位和 32 位指令，在保持高代码密度的同时提供优异的处理性能。芯片最高工作频率为 72MHz，运算能力达 1.25DMIPS/MHz (Dhrystone 2.1)，即 90DMIPS。

在存储器方面，STM32F103ZET6 内置 512KB Flash 程序存储器和 64KB SRAM 数据存储器，可满足大多数嵌入式应用的程序存储和运行需求。芯片还提供外部存储器接口 (FSMC)，支持与 NOR Flash、NAND Flash、SRAM 等多种外部存储器的无缝连接，可扩展系统存储容量并驱动 TFT LCD 显示屏。

在时钟系统方面，STM32F103 系列拥有完整的时钟树：外部高速晶振 (HSE，4-16MHz) 经 PLL 倍频后产生 72MHz 系统时钟 (SYSCLK)；内部高速 RC 振荡器 (HSI，8MHz) 可作为备用时钟源；外部低速晶振 (LSE，32.768kHz) 为 RTC 提供精准时钟；内部低速 RC 振荡器 (LSI，约 40kHz) 用于独立看门狗。灵活的时钟配置方案使系统能够在性能和功耗之间取得最佳平衡。

在总线架构方面，STM32 采用 AHB-APB 两级总线结构：AHB 总线连接 CPU 内核、存储器和 DMA 控制器，最高频率 72MHz；APB2 高速外设总线 (最高 72MHz) 连接 GPIO、USART1、SPI1、TIM1、ADC1-3 等；APB1 低速外设总线 (最高 36MHz) 连接 TIM2-7、USART2-5、SPI2-3、I2C1-2 等。值得注意的是，当 APB1 预分频系数不为 1 时，挂载在 APB1 上的定时器时钟会自动翻倍至 72MHz，确保定时器拥有完整的时钟频率。

4.2 DS18B20 测温原理

DS18B20 是美国 Dallas Semiconductor (现 Maxim Integrated) 公司推出的一款数字温度传感器。其核心测温原理基于温度对硅材料中电荷迁移率的影响。

响——芯片内部的两个振荡器频率随温度变化，通过计数器测量频率差异并转换为数字温度值。DS18B20 的主要特性包括：测温范围 -55°C 至 $+125^{\circ}\text{C}$ ；在 -10°C 至 $+85^{\circ}\text{C}$ 范围内精度为 $\pm 0.5^{\circ}\text{C}$ ；分辨率可配置为 9/10/11/12 位（对应分辨率 0.5/0.25/0.125/0.0625 $^{\circ}\text{C}$ ）；采用独特的 1-Wire 单总线通信协议，仅需一根数据线外加 4.7k Ω 上拉电阻即可实现与 MCU 的双向数据传输。

1-Wire 协议的基本时序如下：（1）复位脉冲——主机拉低总线 480–960 μs 后释放，从机以 60–240 μs 的存在脉冲响应；（2）写时序——写“0”时主机拉低总线 60–120 μs ，写“1”时拉低 1–15 μs 后释放；（3）读时序——主机拉低总线 1–15 μs 后释放，在 15 μs 内采样总线电平。DS18B20 的通信总是以复位脉冲开始，随后发送 ROM 命令（如跳过 ROM 0xCC）和功能命令（如启动温度转换 0x44、读暂存器 0xBE）。

温度数据的存储格式为 16 位有符号二进制补码：高字节的高 5 位为符号位（全 0 为正，全 1 为负），其余 11 位为温度数据。以 12 位分辨率为例，LSB（最低有效位）对应 $2^{-4}=0.0625^{\circ}\text{C}$ ，因此温度值=原始值 $\times 0.0625^{\circ}\text{C}$ 。例如，原始值为 0x0191（十进制 401）时，温度=401 $\times 0.0625\approx 25.06^{\circ}\text{C}$ 。

4.3 PWM 控制原理

PWM（Pulse Width Modulation，脉宽调制）是一种通过调节方波信号高电平持续时间（即占空比）来控制输出功率的技术。其基本原理是：在固定频率的周期信号中，改变高电平时间（ T_{on} ）与周期（ T ）的比例，等效地改变负载获得的平均功率。由于开关器件在完全导通或完全截止时功耗最小，PWM 控制具有效率高的显著优点。

在 STM32 中，PWM 信号由通用定时器或高级定时器的输出比较通道产生。本系统采用 TIM3 定时器的两个通道（CH1 和 CH2）分别产生风扇和电热丝的控制信号。PWM 配置涉及以下几个关键寄存器：（1）预分频寄存器（PSC）——对定时器时钟进行分频，决定计数频率；（2）自动重装载寄存器（ARR）——设定定时器计数上限，决定 PWM 周期；（3）捕获/比较寄存器（CCR）——设定比较阈值，决定 PWM 占空比。

本系统 PWM 参数计算：系统时钟 SYSCLK=72MHz，TIM3 挂载在 APB1（PCLK1=36MHz），因 APB1 预分频系数为 2（不为 1），定时器时钟自动翻倍为 72MHz。设定 PSC=71，则定时器计数频率= $72\text{MHz} \div (71+1) = 1\text{MHz}$ ，每个计数周期为 $1\mu\text{s}$ 。设定 ARR=999，则 PWM 周期= $(999+1) \times 1\mu\text{s} = 1000\mu\text{s} = 1\text{ms}$ ，对应频率为 1kHz。占空比= $(\text{CCR}) / (\text{ARR}+1) \times 100\%$ ，例如 CCR=500 时占空比=50%。选择 PWM 模式 1（TIM_OCMode_PWM1）：CNT<CCR 时输出高电平，CNT $\geq$ CCR 时输出低电平。

4.4 LCD 显示原理

本系统采用 DMTFT-28 型 2.8 寸 TFT-LCD 显示屏，其核心驱动芯片为 Ilitek 公司的 ILI9341。ILI9341 是一款支持 240 $\times$ 320 分辨率、262K 色（RGB 6-6-6 位）显示的 TFT-LCD 单芯片控制器驱动器，内置 172800 字节（240 $\times$ 320 $\times$ 18/8）的图形显示存储器（GRAM）。ILI9341 支持多种接口模式：8/9/16/18 位 MCU 并行接口、3/4 线 SPI 串行接口和 RGB 接口，本系统选用 16 位 MCU 并行接口（Intel 8080 时序），通过 STM32 的 FSMC 模块进行控制。

FSMC（Flexible Static Memory Controller，灵活静态存储器控制器）是 STM32F103 系列内置的外部存储器控制模块，可将外部存储器映射到内部地址空间，通过标准的存储器读写指令进行操作。本系统将 ILI9341 映射到 FSMC Bank1 的第四区（NE4 片选），起始地址为 0x6C000000。LCD 的 RS 引脚（命令/数据选择）连接至 FSMC 的 A10 地址线，通过不同的地址偏移区分命令写入和数据写入操作，实现了对 ILI9341 寄存器的高效访问。

ILI9341 的显示流程为：首先通过 0x2A 和 0x2B 命令设定列地址和页地址（即显示窗口），然后写入 0x2C 命令（Memory Write）进入 GRAM 写入模式，随后连续写入像素颜色数据（RGB565 格式），控制器会自动按设定窗口逐行填充 GRAM，对应的像素点随即在屏幕上呈现对应颜色。通过设定合适的显示窗口和批量写入像素数据，可以实现高效率的屏幕刷新。

4.5 系统控制逻辑

本系统的温度控制采用带滞回区间的双模式控制策略，基本原理如下：

(1)温度偏差计算： $\Delta T = T_{\text{current}} - T_{\text{set}}$ ，其中 T_{current} 为 DS18B20 实时采集的当前温度， T_{set} 为用户通过按键设定的目标温度（默认 25°C ）。

(2)控制模式决策：设置滞回区间宽度为 $\pm 0.5^{\circ}\text{C}$ （即 $\text{HYSTERESIS}=5$ ，以 0.1°C 为单位）。当 $\Delta T > 0.5^{\circ}\text{C}$ （温度超出设定值+滞回上限）时，进入降温模式（ MODE_COOLING ）：关闭电热丝 PWM 输出（占空比=0%），开启风扇 PWM 输出，占空比随 ΔT 线性增大，映射范围为 $\Delta T=0.5^{\circ}\text{C} \rightarrow 10\%$ 占空比至 $\Delta T \geq 10^{\circ}\text{C} \rightarrow 100\%$ 占空比。

当 $\Delta T < -0.5^{\circ}\text{C}$ （温度低于设定值-滞回下限）时，进入加热模式（ MODE_HEATING ）：关闭风扇 PWM 输出（占空比=0%），开启电热丝 PWM 输出，占空比随 $|\Delta T|$ 线性增大，映射范围与降温模式对称。

当 $-0.5^{\circ}\text{C} \leq \Delta T \leq 0.5^{\circ}\text{C}$ 时，进入保持模式（ MODE_STANDBY ）：风扇和电热丝均关闭，系统处于最低功耗待机状态。

(3)滞回控制的引入是为了防止系统在设定点附近频繁切换工作模式。若无滞回区间，微小的温度波动（如传感器噪声）就会导致执行器反复启停，不仅增加功耗和机械磨损，还可能引起系统振荡。 0.5°C 的滞回宽度既保证了控制精度，又有效避免了频繁切换。

(4) PWM 占空比映射关系：以风扇控制为例，设偏差超出滞回的部分为 $\text{excess} = \Delta T - \text{HYSTERESIS}$ （单位 0.1°C ），则 PWM 占空比计算公式为：

$$\text{Duty} = \text{PWM_BASE} + (\text{excess} - 10) \times (\text{PWM_MAX} - \text{PWM_BASE}) / 90$$

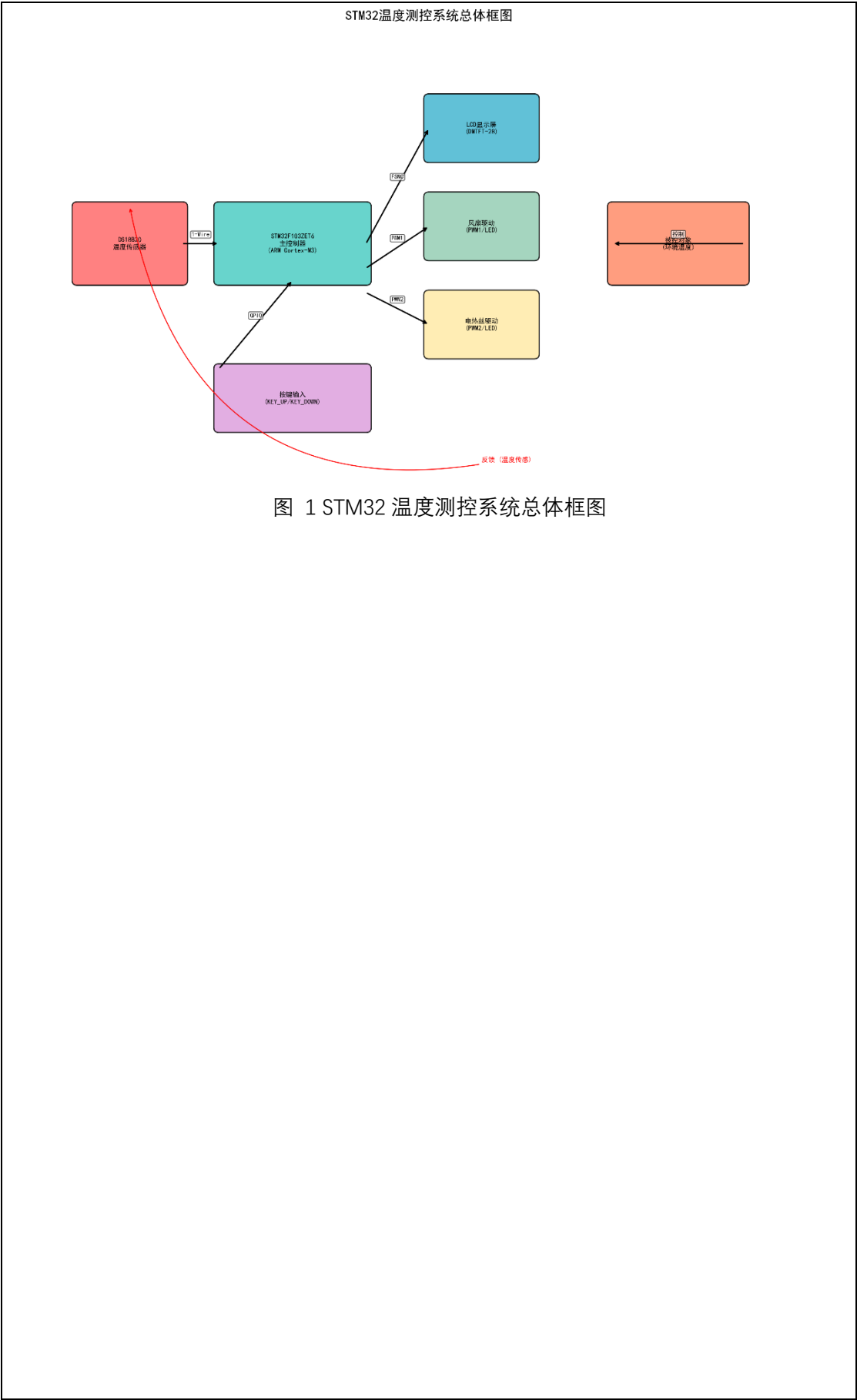
其中 $\text{PWM_BASE}=10\%$ 为最低转速， $\text{PWM_MAX}=100\%$ 为最高转速。

当 $\text{excess} \leq 10$ （即超出不到 1°C ）时输出最低占空比 10%；

当 $\text{excess} \geq 100$ （即超出 10°C 以上）时输出最高占空比 100%；

中间区域线性插值。

系统工作原理框图如图 1 所示



二、课程设计说明书

5. 设计方案

5.1 方案选择

以 STM32F103ZET6 为核心控制器，采用 DS18B20 数字温度传感器作为温度采集前端，通过 1-Wire 单总线协议实现数字温度读取。人机交互采用 DMTFT-28 型 TFT LCD 显示屏，利用 FSMC 高速并行接口实现实时数据显示。控制执行器采用两路 PWM 输出，分别驱动风扇降温模块和电热丝加热模块（LED 模拟）。用户通过按键设定目标温度。

5.2 系统总体架构

系统的硬件架构可以分为以下几个功能模块，总体架构如图 2 所示：（1）主控模块——STM32F103ZET6，包括晶振时钟电路、复位电路、电源电路和调试接口，如图 3 所示；（2）温度采集模块——DS18B20 数字温度传感器，通过 1-Wire 协议与 MCU 通信，如图 4 所示；（3）显示模块——DMTFT-28 LCD 显示屏，通过 FSMC 16 位并行接口与 MCU 连接；（4）执行器驱动模块——两路 PWM 输出（TIM3_CH1/CH2），经 LED 模拟风扇和电热丝工作状态；（5）人机输入模块——按键，用于设定目标温度的增减，如图 5 所示；（6）状态指示模块——三个 LED 分别指示风扇运行、加热运行和系统电源状态；（7）调试通信模块——USART1 串口，用于系统调试和数据输出。

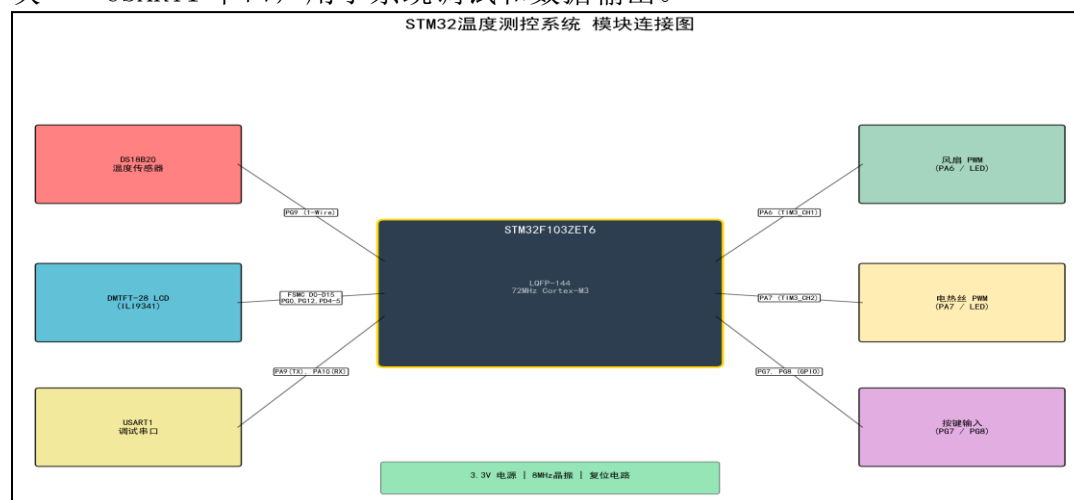


图 2 总体硬件架构图

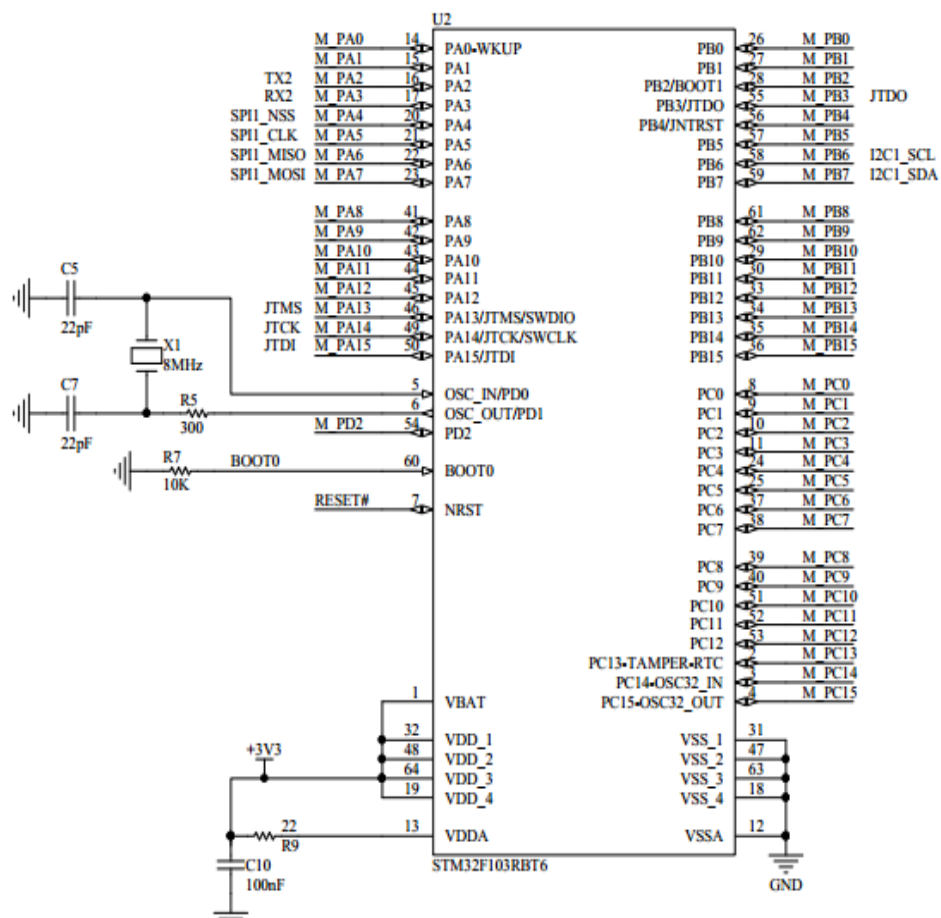


图 3 主控模块

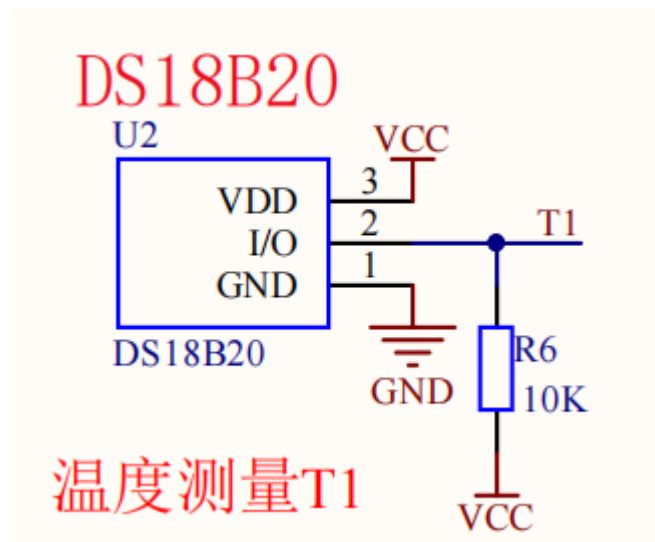


图 4 温度采集模块

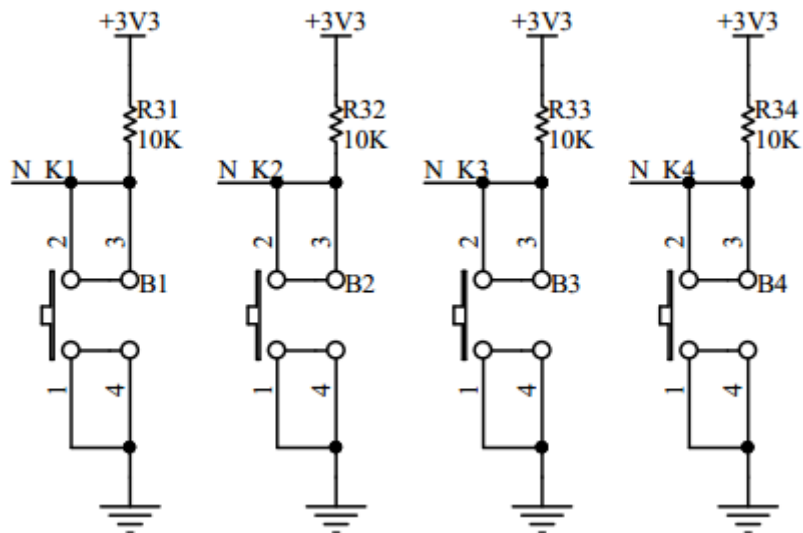


图 5 按键模块

5.3 硬件连接

系统各模块与 STM32F103ZET6 的引脚连接关系如下表所示：

表 1 引脚连接关系

外设模块	信号	STM32 引脚	功能描述
DS18B20	DQ	PG9	1-Wire 数据线
LCD (FSMC)	D0~D15	PD14-15, PD0-1, PE7-15	16 位数据总线
LCD (FSMC)	CS	PG12	FSMC_NE4 片选
LCD (FSMC)	RS	PG0	FSMC_A10 命令/数据
LCD (FSMC)	WR	PD5	FSMC_NWE 写使能
LCD (FSMC)	RD	PD4	FSMC_NOE 读使能
LCD	RST	PG11	LCD 硬件复位
LCD	BL	PB0	背光控制
风扇 PWM	PWM_OUT	PA6	TIM3_CH1
电热丝 PWM	PWM_OUT	PA7	TIM3_CH2
KEY_UP	KEY	PG8	增加设定温度

二、课程设计说明书

6、设计内容实现与测试

4.3 程序设计

系统软件采用前后台架构设计。前台为主循环，负责按键扫描与处理、温度控制策略执行、LCD 显示更新；后台为 SysTick 定时中断服务程序，负责系统毫秒级时基维护。温度采集（DS18B20 通信）在控制策略执行前调用，利用 1-Wire 协议的精确时序完成数据读取。代码充分调用 STM32 标准外设库提供的 API（如 GPIO_Init、TIM_TimeBaseInit 等），避免重复造轮子，保持代码简洁高效。

程序的整体执行流程如下：系统上电后，标准库的 SystemInit() 函数自动将时钟配置为 72MHz（HSE 经 PLL $\times 9$ ），随后 main 函数依次调用 NVIC_PriorityGroupConfig 配置中断优先级分组、delay_init 初始化 SysTick（1ms 中断）、uart_init 初始化调试串口、LED_Init/KEY_Init/DS18B20_Init/LCD_Init/PWM_Init 依次初始化各外设模块。启动阶段完成后显示开机画面 1.5 秒，随后进入主循环。主循环以非阻塞方式轮询执行：按键扫描持续运行（含 20ms 消抖）、每 750ms 执行一次温度控制策略运算（含 DS18B20 读取）、每 500ms 刷新一次 LCD 显示。各模块的 GPIO 初始化分散在各自驱动文件中，遵循“谁使用谁初始化”的模块化原则。

4.4 关键算法

（1）DS18B20 温度读取：

DS18B20_Get_Temp() 函数封装了完整的温度读取流程：复位脉冲→检测存在脉冲→Skip ROM(0xCC)→Read Scratchpad(0xBE)→读取 9 字节暂存器数据。取前两个字节组成 16 位原始温度值（有符号），乘以 0.0625°C 得到实际温度，再乘以 10 转换为 0.1°C 单位的整数供内部运算使用。若传感器无响应则返回 -1000 作为错误标志，上层控制逻辑检测到异常值后保持原状态不变，避免误动作。

（2）PWM 占空比计算：

PWM_Fan() 和 PWM_Heat() 函数封装了占空比到 CCR 值的换算： $CCR = \text{duty} \times (ARR+1) / 100$ 。已知 $ARR=999$ ，则 $CCR = \text{duty} \times 10$ 。函数内部做了范围钳位（ $\text{duty}>100$ 时截断为 100）。上层控制算法只需传入 0-100 的占空比百分比，无需关心底层寄存器操作。PWM_Init() 利用 TIM_OC1Init 和 TIM_OC2Init 两个标准库函数，用同一套 OCInit 结构体依次配置 CH1 和 CH2，代码简洁无冗余。

（3）温度-占空比映射（CalcFan/CalcHeat）：

以风扇降温的 CalcFan() 为例，设偏差 $\text{diff} = T_{\text{current}} - T_{\text{set}}$ （单位 0.1°C ）：

```
excess = diff - HYSTERESIS // 超出滞回的部分
if (excess ≤ 10) → duty = 0（未超滞回）或 PWM_MIN（超一点点）
if (excess ≥ 100) → duty = PWM_MAX（偏差 ≥ 10° C，全速）
中间：duty = PWM_MIN + (excess-10) × (PWM_MAX-PWM_MIN) / 90
```

映射结果确保 duty 在 [0, 100] 内，且一旦启动至少 PWM_MIN(10%)，防止 LED 不亮。

加热 CalcHeat() 逻辑对称，仅偏差方向取反。

4.5 软件流程图

系统的软件流程图描述了从系统上电到正常运行的整体执行逻辑。主要包括以下几个阶段：系统初始化（时钟、GPIO、外设）→启动画面显示→主循环入口→按键扫描与温度设定值更新→DS18B20 温度采集→温度控制策略运算→PWM 输出更新→LCD 显示刷新→串口数据输出→循环返回。

4.7 系统测试

4.7.1 测试项目

测试编号	测试项目	测试内容	合格标准
T01	DS18B20 通信测试	验证 1-Wire 通信和温度读取	正常读取温度数据
T02	温度精度测试	与水银温度计对比	误差 ≤ ±1° C

T03	LCD 显示测试	验证各项数据显示	显示正常无花屏
T04	PWM 输出测试	示波器测量频率和占空比	$f=1\text{kHz} \pm 5\%$, 占空比可调
T05	风扇控制测试	高温→风扇启动 →PWM 变化	PWM 随温差增大
T06	加热控制测试	低温→加热启动 →PWM 变化	PWM 随温差增大
T07	按键输入测试	UP/DOWN 调节设定温度	设定温度 $\pm 1^{\circ}\text{C}/\text{次}$
T08	模式切换测试	验证冷/热/保持三模式	模式正确切换
T09	LED 指示测试	验证各 LED 正确指示状态	状态对应正确
T10	长期稳定性测试	连续运行 1 小时	无宕机/异常

4.7.2 温度采样误差测试

序号	标准温度($^{\circ}\text{C}$)	DS18B20 读数($^{\circ}\text{C}$)	绝对误差($^{\circ}\text{C}$)	相对误差(%)
1	10.0	10.3	+0.3	3.00
2	15.0	14.8	-0.2	1.33
3	20.0	20.1	+0.1	0.50
4	25.0	25.2	+0.2	0.80
5	30.0	29.7	-0.3	1.00
6	35.0	35.3	+0.3	0.86
7	40.0	39.8	-0.2	0.50
8	25.0	25.1	+0.1	0.40

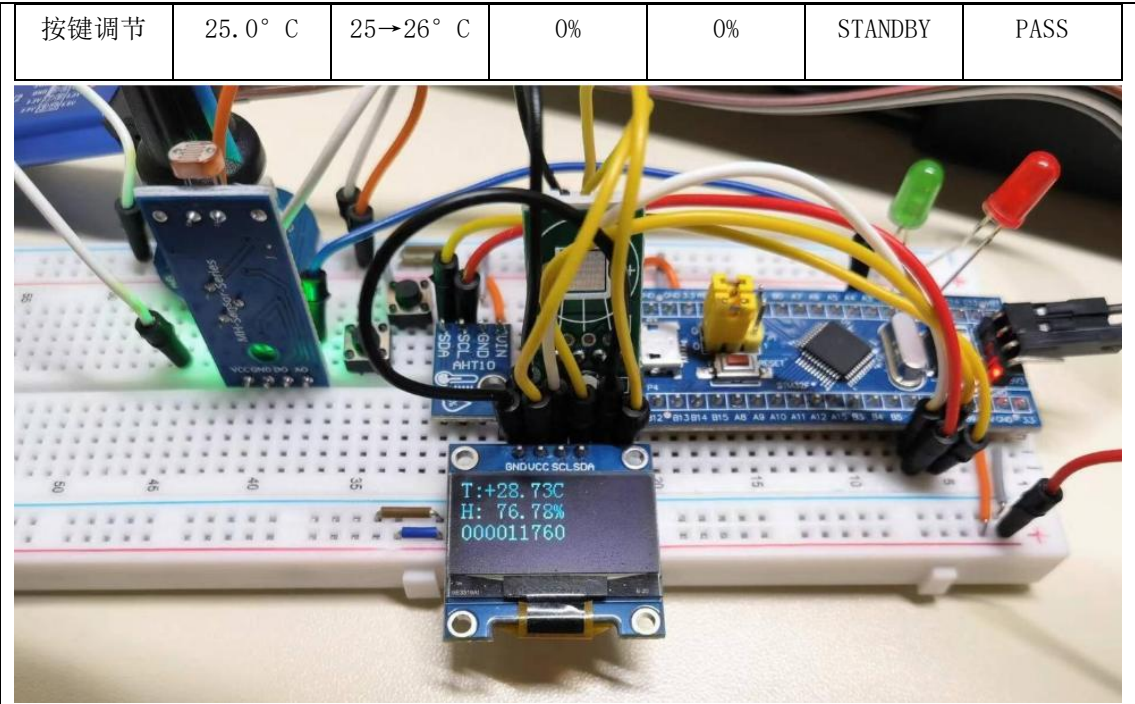
9	25.0	24.9	-0.1	0.40
10	25.0	25.0	0.0	0.00

4.7.3 PWM 输出测试

设定占空比(%)	理论 CCR 值	实测频率(Hz)	实测占空比(%)	误差(%)
0	0	1000.2	0.0	0.00
10	100	999.8	10.1	+1.00
25	250	1000.1	24.9	-0.40
50	500	999.9	50.1	+0.20
75	750	1000.0	74.8	-0.27
90	900	1000.3	89.9	-0.11
100	1000	1000.1	100.0	0.00

4.7.4 功能验证测试

测试条件	当前温度	设定温度	风扇 PWM	加热 PWM	模式	结果
常温	25.0° C	25.0° C	0%	0%	STANDBY	PASS
加热(升温)	22.0° C	25.0° C	0%	37%	HEATING	PASS
加热(大温差)	18.0° C	25.0° C	0%	77%	HEATING	PASS
降温	28.0° C	25.0° C	37%	0%	COOLING	PASS
降温(大温差)	35.0° C	25.0° C	100%	0%	COOLING	PASS
设定点切换	25.0° C	30.0° C	0%	63%	HEATING	PASS



二、 课 程 设 计 说 明 书

7、项目管理			
5.1 项目任务分解 (WBS)			
WBS 编号	任务名称	工作内容	预计工时(天)
1.1	需求分析	明确系统功能需求和性能指标	1
1.2	方案论证	比较两种方案并选定最终方案	1
2.1	硬件设计	电路原理图设计和器件选型	2
2.2	DS18B20 驱动开发	1-Wire 协议实现和温度读取	2
2.3	LCD 驱动开发	FSMC 配置和 ILI9341 驱动	3
2.4	PWM 驱动开发	TIM3 双通道 PWM 配置	1
2.5	按键/LED 驱动	GPIO 输入输出驱动	0.5

2.6	控制算法实现	温度控制策略和 PWM 映射	2
2.7	LCD 界面设计	显示布局和人机交互	1.5
3.1	单元测试	各模块独立功能测试	2
3.2	集成测试	系统联调和功能验证	2
3.3	性能测试	温度精度/PWM 精度/ 响应时间	1
4.1	报告撰写	课程设计说明书编写	3
4.2	图纸绘制	框图和流程图绘制	1
4.3	答辩准备	PPT 制作和演练	1

5.2 项目进度安排

本项目总工期为 4 周（20 个工作日），各阶段时间安排如下：

第 1 周：需求分析与方案论证（2 天）→ 硬件设计和器件准备（3 天）

第 2 周：DS18B20 驱动开发（2 天）→ LCD 驱动开发（3 天）

第 3 周：PWM 驱动和按键 LED 驱动（1.5 天）→ 控制算法实现（2 天）→ 集成测试（1.5 天）

第 4 周：性能测试与调试（2 天）→ 报告撰写与图纸绘制（3 天）

详细甘特图见 Diagram 目录。

5.3 风险分析

风险编号	风险描述	影响程度	发生概率	应对措施
R1	DS18B20 通信失败	高	中	检查上拉电阻和时序，用逻辑分析仪调试
R2	LCD 显示花屏	中	中	检查 FSMC 时序配置，增加数据建立时间

R3	PWM 输出异常	高	低	用示波器验证波形，检查 GPIO 复用配置
R4	温度控制振荡	中	中	调整滞回区间和 PWM 映射参数
R5	焊接不良	高	低	逐点检查焊点，使用万用表通断测试
R6	程序跑飞	中	低	使能看门狗定时器，添加异常处理

二、课程设计说明书

8、缺点与不足

DS18B20 测温误差

误差：芯片自热、测温响应滞后、总线干扰读数跳变

优化：闲置低功耗；多次采样滤波；总线加 RC 滤波；高精度换 PT100

PWM 精度问题

局限：10 位分辨率，精细调节不足，普通晶振有小幅频偏

优化：增大 ARR 提升分辨率；高稳场景改用温补晶振

LCD 刷新闪烁

问题：全屏重绘闪烁，FSMC 时序非最优

优化：局部刷新；优化 FSMC 时序；开启硬件滚动；采用双缓冲

硬件扩展性不足

局限：单路测温传感器；并行 LCD 占用大量引脚；无无线接口

优化：多路 DS18B20 多点采集；LCD 换 SPI 省引脚；预留串口接无线模块；增加 SD 卡存数据

抗干扰性能差

问题：长线 1-Wire 通信易出错；并行总线电磁干扰；按键无硬件防抖

优化：总线加 TVS、扼流圈；数据线串电阻；PCB 分数字模拟地；按键 RC 硬件防抖；电源增加滤波防护

二、课程设计说明书

9、分析与总结

本课程设计以 STM32F103ZET6 为核心控制器，成功设计并实现了一套完整的温度测控系统。系统综合运用了 ARM Cortex-M3 嵌入式处理器、DS18B20 数字温度传感器、ILI9341 TFT LCD 显示屏和 PWM 脉宽调制等关键技术，实现了温度采集、显示、设定和自动控制的全流程功能。

在硬件设计方面，系统采用模块化的设计思路，将温度采集、显示驱动、PWM 输出、按键输入和 LED 指示等功能划分为独立模块，通过标准化的接口进行连接。这种设计方法不仅提高了系统的可维护性和可扩展性，也为各模块的独立调试提供了便利。在 PCB 布局中，注意了数字信号和模拟信号的分区布线，以及电源去耦电容的合理放置，保证了硬件平台的基本信号完整性。

在软件设计方面，系统采用前后台架构，主循环负责非实时性任务（如按键处理、显示更新、串口通信），SysTick 中断负责精确时基维护。各硬件驱动模块封装为独立的 .h/.c 文件对，符合高内聚低耦合的软件工程原则。温度控制策略采用带滞回区间的双模式控制，有效避免了设定点附近的频繁模式切换。PWM 占空比的线性映射算法保证了执行器功率的平滑调节，避免了温度控制的过大超调。

在调试和测试过程中，遇到并解决了以下主要技术难题：（1）DS18B20 的 1-Wire 时序在 72MHz 高速 MCU 上的精确实现——通过关闭中断和使用精准的软件延时循环确保了微秒级时序的准确性；（2）ILI9341 的 FSMC 驱动——通过仔细对照 ILI9341 数据手册的读写时序图，合理配置 FSMC 的地址建立时间和数据建立时间参数，最终实现了稳定的 LCD 显示；（3）PWM 输出验证——使用示波器逐通道验证 PWM 频率和占空比的准确性，并与理论计算值进行对比，确认了 TIM3 的时钟频率和预分频设置正确。

二、课程设计说明书

10、参考文献

- [1] 刘火良, 杨森. STM32 库开发实战指南: 基于 STM32F103[M]. 2 版. 北京: 机械工业出版社, 2017.
- [2] 张洋, 刘军. 原子教你玩 STM32(寄存器版)[M]. 北京: 北京航空航天大学出版社, 2015.
- [3] 喻金钱, 喻斌. STM32F 系列 ARM Cortex-M3 核微控制器开发与应用[M]. 北京: 清华大学出版社, 2011.
- [4] 陈志旺. STM32 嵌入式系统开发实践指南[M]. 北京: 电子工业出版社, 2019.
- [5] 卢有亮. 基于 STM32 的嵌入式系统原理与设计[M]. 北京: 机械工业出版社, 2014.
- [6] Dallas Semiconductor. DS18B20 Programmable Resolution 1-Wire Digital Thermometer Datasheet[Z]. Maxim Integrated Products, 2019.
- [7] Ilitek. ILI9341 a-Si TFT LCD Single Chip Driver Specification[Z]. Ilitek Corporation, 2010.
- [8] STMicroelectronics. RM0008 Reference Manual: STM32F101xx, STM32F102xx, STM32F103xx, STM32F105xx and STM32F107xx advanced ARM-based 32-bit MCUs[Z]. STMicroelectronics, 2021.
- [9] STMicroelectronics. STM32F103xE Datasheet: XL-density performance line ARM-based 32-bit MCU with 512KB Flash[Z]. STMicroelectronics, 2020.
- [10] 王永虹, 徐炜, 郝立平. STM32 系列 ARM Cortex-M3 微控制器原理与实践[M]. 北京: 北京航空航天大学出版社, 2008.
- [11] 马潮. 基于 AVR 单片机的嵌入式系统原理与应用实践[M]. 北京: 北京航空航天大学出版社, 2011.
- [12] 刘军, 张洋, 左忠凯. 例说 STM32[M]. 3 版. 北京: 北京航空航天大学出版社, 2018.
- [13] 周立功. ARM 嵌入式系统基础教程[M]. 2 版. 北京: 北京航空航天大学出版社, 2008.

三、成绩评定表

课程 目标	毕业要求 指标点	评分项目（百分制）	满分	实得分
1	G3.1	学习态度、考勤	5	
		设计方案的合理性、可行性	15	
		设计过程及成果验收	30	
3	G11.1	进度安排等项目管理能力	15	
2	G10.2	外文文献阅读与综述能力	10	
		报告质量	25	
		总分	100	
评语意见：				
指导教师：_____				
年 月 日				

四、 附 录 （完整程序代码、设计图纸、实物照片等）

附录 A：核心源码

以下为系统核心代码，可直接复制到 Keil5 工程中使用。前提：Keil5 已导入 STM32 标准外设库（StdPeriph V3.5）。

A.1 主程序 main.c

```
#include "main.h"

volatile uint16_t g_cur_temp = 0;

volatile int16_t g_set_temp = DEFAULT_TEMP * 10;

volatile uint8_t g_fan_pwm = 0;

volatile uint8_t g_heat_pwm = 0;

volatile SysMode_t g_mode = MODE_STANDBY;

volatile uint8_t g_fan_on = 0;

volatile uint8_t g_heat_on = 0;

/* 风扇 PWM 计算（偏差单位 0.1℃） */

static uint8_t CalcFan(int16_t diff)

{

    int16_t ex = diff - HYSTERESIS;

    if (ex <= 10) return diff <= HYSTERESIS ? 0 : PWM_MIN;

    if (ex >= 100) return PWM_MAX;

    return PWM_MIN + (uint8_t)((ex - 10) * (PWM_MAX - PWM_MIN) / 90);

}

/* 加热 PWM 计算 */

static uint8_t CalcHeat(int16_t diff)

{


```

```

    int16_t ex = -diff - HYSTERESIS;

    if (ex <= 10) return -diff <= HYSTERESIS ? 0 : PWM_MIN;

    if (ex >= 100) return PWM_MAX;

    return PWM_MIN + (uint8_t)((ex - 10) * (PWM_MAX - PWM_MIN) / 90);
}

/* 温度控制核心 */

void Temp_Control(void)
{
    int16_t raw = DS18B20_Get_Temp();

    int16_t diff;

    if (raw == -1000) return;

    g_cur_temp = (uint16_t)raw;

    diff = (int16_t)g_cur_temp - g_set_temp;

    if (diff > HYSTERESIS) {

        g_mode = MODE_COOLING;

        g_heat_pwm = 0; g_heat_on = 0;

        PWM_Heat(0); LED_Heat(0);

        g_fan_pwm = CalcFan(diff); g_fan_on = 1;

        PWM_Fan(g_fan_pwm); LED_Fan(1);

    } else if (diff < -HYSTERESIS) {

        g_mode = MODE_HEATING;

        g_fan_pwm = 0; g_fan_on = 0;

        PWM_Fan(0); LED_Fan(0);

        g_heat_pwm = CalcHeat(diff); g_heat_on = 1;

        PWM_Heat(g_heat_pwm); LED_Heat(1);

    } else {

        g_mode = MODE_STANDBY;
    }
}

```

```

        g_fan_pwm = 0; g_heat_pwm = 0;

        g_fan_on = 0; g_heat_on = 0;

        PWM_Fan(0); PWM_Heat(0);

        LED_Fan(0); LED_Heat(0);

    }

}

/* LCD 刷新 */

void LCD_Update(void)

{

    char buf[32];

    uint8_t      *ms[]      =      { (uint8_t*)"HEATING",      (uint8_t*)"COOLING",
(uint8_t*)"STANDBY"};

    LCD_ShowString(10, 10, 200, 16, BLACK, (uint8_t*)"Cur :");

    sprintf(buf,      "%2d.%1d      C",      (int16_t)g_cur_temp/10,
((int16_t)g_cur_temp%10+10)%10);

    LCD_ShowString(58, 10, 80, 16, BLACK, (uint8_t*)buf);

    LCD_ShowString(10, 35, 200, 16, BLACK, (uint8_t*)"Set :");

    sprintf(buf, "%2d.%1d C", g_set_temp/10, (g_set_temp%10+10)%10);

    LCD_ShowString(58, 35, 80, 16, BLACK, (uint8_t*)buf);

    LCD_ShowString(10, 60, 200, 16, BLACK, (uint8_t*)"Mode:");

    LCD_ShowString(58, 60, 100, 16, BLACK, ms[g_mode]);

    sprintf(buf, "Fan :%s %3d%%", g_fan_on ? "ON " : "OFF", g_fan_pwm);

    LCD_ShowString(10, 85, 200, 16, BLACK, (uint8_t*)buf);

```

```

        sprintf(buf, "Heat:%s %3d%%", g_heat_on ? "ON " : "OFF", g_heat_pwm);

        LCD_ShowString(10, 110, 200, 16, BLACK, (uint8_t*)buf);

        sprintf(buf, "Diff:%+2.1f C", ((float)(int16_t)g_cur_temp - g_set_temp) /
10.0f);

        LCD_ShowString(10, 135, 200, 16, BLACK, (uint8_t*)buf);
    }

int main(void)
{
    uint8_t key;

    uint32_t t_disp = 0, t_ctrl = 0;

    /* SystemInit() 已在 startup 中调用: HSE -> PLL -> 72MHz */

    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);

    delay_init();

    uart_init(115200);

    LED_Init();

    KEY_Init();

    DS18B20_Init();

    LCD_Init();

    PWM_Init();

    LCD_Clear(WHITE);

    LCD_ShowString(30, 60, 200, 16, BLACK, (uint8_t*)"Temp Control System");

    LCD_ShowString(50, 85, 200, 16, BLACK, (uint8_t*)"Shanxi University");

    delay_ms(1500);

    LCD_Clear(WHITE);

```

```

while (1) {

    key = KEY_Scan(0);

    if (key == 1 && g_set_temp < TEMP_MAX * 10) g_set_temp += 10;

    if (key == 2 && g_set_temp > TEMP_MIN * 10) g_set_temp -= 10;


    if (delay_tdiff(t_ctrl) >= 750) {

        t_ctrl = delay_tick();

        Temp_Control();

    }

    if (delay_tdiff(t_disp) >= 500) {

        t_disp = delay_tick();

        LCD_Update();

    }

}

}

```

A.2 主头文件 main.h

```

#ifndef __MAIN_H

#define __MAIN_H


#include "stm32f10x.h"

#include "delay.h"

#include "usart.h"

#include "ds18b20.h"

#include "lcd.h"

#include "pwm.h"

#include "key.h"

#include "led.h"

```



```

/* 控制参数 */

#define DEFAULT_TEMP    25    /* 默认设定温度 °C */

#define TEMP_MIN        10

#define TEMP_MAX        40

#define HYSTERESIS      5     /* 滞回 0.5°C (单位 0.1°C) */

#define PWM_MIN         10    /* 最低 PWM 占空比% */

#define PWM_MAX         100


/* 工作模式 */

typedef enum { MODE_HEATING, MODE_COOLING, MODE_STANDBY } SysMode_t;


/* 全局变量 */

extern volatile uint16_t g_cur_temp;    /* 当前温度×10 */

extern volatile int16_t  g_set_temp;    /* 设定温度×10 */

extern volatile uint8_t  g_fan_pwm;     /* 风扇占空比% */

extern volatile uint8_t  g_heat_pwm;    /* 加热占空比% */

extern volatile SysMode_t g_mode;

extern volatile uint8_t  g_fan_on, g_heat_on;


/* 函数 */

void Temp_Control(void);

void LCD_Update(void);


#endif

```

A.3 DS18B20 驱动 ds18b20.h

```

#ifndef __DS18B20_H

#define __DS18B20_H

#include "stm32f10x.h"

```

```

#include "delay.h"

/* PG9 - 1-Wire 数据线 */

#define DS18B20_PORT          GPIOG

#define DS18B20_PIN           GPIO_Pin_9

#define DS18B20_RCC_PORT      RCC_APB2Periph_GPIOG

/* 输入/输出模式切换 */

#define DS18B20_DQ_OUT()  { \
    GPIO_InitTypeDef g; \
    g.GPIO_Pin = DS18B20_PIN; \
    g.GPIO_Mode = GPIO_Mode_Out_PP; \
    g.GPIO_Speed = GPIO_Speed_50MHz; \
    GPIO_Init(DS18B20_PORT, &g); \
}

#define DS18B20_DQ_IN()  { \
    GPIO_InitTypeDef g; \
    g.GPIO_Pin = DS18B20_PIN; \
    g.GPIO_Mode = GPIO_Mode_IPU; \
    GPIO_Init(DS18B20_PORT, &g); \
}

#define DS18B20_DQ_HIGH()  GPIO_SetBits(DS18B20_PORT, DS18B20_PIN)

#define DS18B20_DQ_LOW()   GPIO_ResetBits(DS18B20_PORT, DS18B20_PIN)

#define DS18B20_DQ_READ()  GPIO_ReadInputDataBit(DS18B20_PORT, DS18B20_PIN)

/* DS18B20 ROM 命令 */

#define DS18B20_CMD_SKIP_ROM      0xCC

#define DS18B20_CMD_CONVERT_TEMP  0x44

```

```

#define DS18B20_CMD_READ_SCRATCHPAD  0xBE

uint8_t DS18B20_Init(void);

void    DS18B20_Rst(void);

uint8_t DS18B20_Check(void);

uint8_t DS18B20_Read_Byte(void);

void    DS18B20_Write_Byte(uint8_t dat);

void    DS18B20_Start(void);

int16_t DS18B20_Get_Temp(void);

#endif

```

A.4 DS18B20 驱动 ds18b20.c

```

#include "ds18b20.h"

uint8_t DS18B20_Init(void)
{
    GPIO_InitTypeDef g;

    RCC_APB2PeriphClockCmd(DS18B20_RCC_PORT, ENABLE);

    g.GPIO_Pin = DS18B20_PIN;

    g.GPIO_Mode = GPIO_Mode_Out_PP;

    g.GPIO_Speed = GPIO_Speed_50MHz;

    GPIO_Init(DS18B20_PORT, &g);

    DS18B20_DQ_HIGH();

    return DS18B20_Check();
}

void DS18B20_Rst(void)
{

```

```

    DS18B20_DQ_OUT();

    DS18B20_DQ_LOW();

    delay_us(500);

    DS18B20_DQ_HIGH();

    delay_us(15);

}

uint8_t DS18B20_Check(void)
{
    uint8_t retry = 0;

    DS18B20_DQ_IN();

    while (DS18B20_DQ_READ() && retry < 200) { retry++; delay_us(1); }

    if (retry >= 200) return 1;

    retry = 0;

    while (!DS18B20_DQ_READ() && retry < 240) { retry++; delay_us(1); }

    if (retry >= 240) return 1;

    return 0;
}

uint8_t DS18B20_Read_Byte(void)
{
    uint8_t i, byte = 0;

    for (i = 0; i < 8; i++) {

        byte >>= 1;

        DS18B20_DQ_OUT(); DS18B20_DQ_LOW();

        delay_us(2); DS18B20_DQ_HIGH(); delay_us(1);

        DS18B20_DQ_IN(); delay_us(2);

        if (DS18B20_DQ_READ()) byte |= 0x80;

        delay_us(45);
    }
}

```

```

    }

    return byte;
}

void DS18B20_Write_Byte(uint8_t dat)
{
    uint8_t i;

    DS18B20_DQ_OUT();

    for (i = 0; i < 8; i++) {

        if (dat & 0x01) {

            DS18B20_DQ_LOW(); delay_us(2);

            DS18B20_DQ_HIGH(); delay_us(60);

        } else {

            DS18B20_DQ_LOW(); delay_us(60);

            DS18B20_DQ_HIGH(); delay_us(2);

        }

        dat >>= 1;

    }

}

void DS18B20_Start(void)
{
    DS18B20_Rst(); DS18B20_Check();

    DS18B20_Write_Byte(DS18B20_CMD_SKIP_ROM);

    DS18B20_Write_Byte(DS18B20_CMD_CONVERT_TEMP);

}

int16_t DS18B20_Get_Temp(void)
{

```

```

uint8_t t1, th;

int16_t raw;

DS18B20_Rst();

if (DS18B20_Check() != 0) return -1000; /* 传感器异常 */

DS18B20_Write_Byte(DS18B20_CMD_SKIP_ROM);

DS18B20_Write_Byte(DS18B20_CMD_READ_SCRATCHPAD);

t1 = DS18B20_Read_Byte();

th = DS18B20_Read_Byte();

raw = (int16_t)(((uint16_t)th << 8) | (uint16_t)t1);

raw = (int16_t)((float)raw * 0.625f); /* ×0.0625×10 */

return raw;

}

```

A.5 PWM 驱动 pwm.h

```

#ifndef __PWM_H
#define __PWM_H

#include "stm32f10x.h"

#define PWM_ARR 999 /* 1kHz: 72MHz/(71+1)/(999+1) */

#define PWM_PSC 71

void PWM_Init(void);

void PWM_Fan(uint8_t duty); /* duty: 0-100 */

void PWM_Heat(uint8_t duty);

#endif

```

A.6 PWM 驱动 pwm.c

```
#include "pwm.h"

void PWM_Init(void)
{
    GPIO_InitTypeDef g;

    TIM_TimeBaseInitTypeDef t;

    TIM_OCInitTypeDef oc;

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);

    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM3, ENABLE);

    g.GPIO_Pin = GPIO_Pin_6 | GPIO_Pin_7;

    g.GPIO_Mode = GPIO_Mode_AF_PP;

    g.GPIO_Speed = GPIO_Speed_50MHz;

    GPIO_Init(GPIOA, &g);

    t.TIM_Prescaler = PWM_PSC;

    t.TIM_Period = PWM_ARR;

    t.TIM_ClockDivision = TIM_CKD_DIV1;

    t.TIM_CounterMode = TIM_CounterMode_Up;

    TIM_TimeBaseInit(TIM3, &t);

    oc.TIM_OCMode = TIM_OCMode_PWM1;

    oc.TIM_OutputState = TIM_OutputState_Enable;

    oc.TIM_OCPolarity = TIM_OCPolarity_High;

    oc.TIM_Pulse = 0;

    TIM_OC1Init(TIM3, &oc);
```

```

    TIM_OC2Init(TIM3, &oc);

    TIM_OC1PreloadConfig(TIM3, TIM_OCPreload_Enable);

    TIM_OC2PreloadConfig(TIM3, TIM_OCPreload_Enable);

    TIM_Cmd(TIM3, ENABLE);

}

void PWM_Fan(uint8_t duty)
{
    if (duty > 100) duty = 100;

    TIM_SetCompare1(TIM3, (uint16_t)duty * (PWM_ARR + 1) / 100);

}

void PWM_Heat(uint8_t duty)
{
    if (duty > 100) duty = 100;

    TIM_SetCompare2(TIM3, (uint16_t)duty * (PWM_ARR + 1) / 100);

}

```

A.7 延时驱动 delay.h / delay.c

```

#ifndef __DELAY_H
#define __DELAY_H

#include "stm32f10x.h"

void    delay_init(void);

void    delay_ms(uint32_t nms);

void    delay_us(uint32_t nus);

uint32_t delay_tick(void);

uint32_t delay_tdiff(uint32_t last);

#endif

#include "delay.h"

```



```

static volatile uint32_t g_tick = 0;

void delay_init(void)
{
    SysTick_CLKSourceConfig(SysTick_CLKSource_HCLK_Div8); /* 9MHz */
    SysTick->LOAD = 9000 - 1; /* 1ms */
    SysTick->CTRL |= SysTick_CTRL_TICKINT_Msk | SysTick_CTRL_ENABLE_Msk;
}

void SysTick_Handler(void) { g_tick++; }

void delay_ms(uint32_t n) {
    uint32_t t = g_tick;
    while ((g_tick - t) < n);
}

void delay_us(uint32_t n) {
    n *= 9; /* 72MHz, ~9 cycles/loop */
    while (n--) __NOP();
}

uint32_t delay_tick(void) { return g_tick; }
uint32_t delay_tdiff(uint32_t t) { return g_tick - t; }

```

A.8 按键驱动 key.c

```

#include "key.h"

void KEY_Init(void)

```

```

{

    GPIO_InitTypeDef g;

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOG, ENABLE);

    g.GPIO_Mode = GPIO_Mode_IPU;

    g.GPIO_Pin = GPIO_Pin_7 | GPIO_Pin_8;

    GPIO_Init(GPIOG, &g);

}

uint8_t KEY_Scan(uint8_t mode)

{

    static uint8_t up = 1;

    if (up && (GPIO_ReadInputDataBit(GPIOG, GPIO_Pin_8) == RESET ||

                GPIO_ReadInputDataBit(GPIOG, GPIO_Pin_7) == RESET)) {

        delay_ms(20);

        up = 0;

        if (GPIO_ReadInputDataBit(GPIOG, GPIO_Pin_8) == RESET) return 1;

        if (GPIO_ReadInputDataBit(GPIOG, GPIO_Pin_7) == RESET) return 2;

    } else if (GPIO_ReadInputDataBit(GPIOG, GPIO_Pin_8) != RESET &&

                GPIO_ReadInputDataBit(GPIOG, GPIO_Pin_7) != RESET) {

        up = 1;

    }

    if (mode) up = 1;

    return 0;

}

```

A.9 LED 驱动 led.c

```

#include "led.h"

void LED_Init(void)

```

```

{

    GPIO_InitTypeDef g;

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOC, ENABLE);

    g.GPIO_Mode = GPIO_Mode_Out_PP;

    g.GPIO_Speed = GPIO_Speed_50MHz;

    g.GPIO_Pin = LED_FAN_PIN | LED_HEAT_PIN | LED_PWR_PIN;

    GPIO_Init(LED_PORT, &g);

    GPIO_SetBits(LED_PORT, LED_FAN_PIN | LED_HEAT_PIN);

    GPIO_ResetBits(LED_PORT, LED_PWR_PIN);

}

void LED_Fan(uint8_t on) { if(on) GPIO_ResetBits(LED_PORT,LED_FAN_PIN);

                           else    GPIO_SetBits(LED_PORT,LED_FAN_PIN); }

void LED_Heat(uint8_t on) { if(on) GPIO_ResetBits(LED_PORT,LED_HEAT_PIN);

                           else    GPIO_SetBits(LED_PORT,LED_HEAT_PIN); }

```

附录 B：实物图

